CUPS Printer Discovery Security Lab Report

1. Environment and architecture

The lab uses the internal Docker network `172.28.77.0/24`. The Vulhub `cups-browsed:2.0.1` victim is `172.28.77.20`; the `ippserver` virtual printer is `172.28.77.30`. The Scapy attacker is deliberately kept outside the container network unless a controlled transmission is being performed.

The complete specification is ../assignment.md. The implementation is described in ../README.md.

2. Implementation

`cups_attack.py` builds an IP/UDP/Raw packet whose destination is UDP/631 and whose payload is the historical browse advertisement format. It accepts custom source, printer address, port, queue, message variation, repeat count, output PCAP, and log paths. `cups_detector.py` parses UDP/631 advertisements, validates the URI and fields, checks advertiser/printer allowlists, detects repeated signatures, and reports TCP/631 endpoints outside the allowlist. It emits JSON or CSV and can consume `rdpcap` input or `sniff` live traffic.

3. Experiment matrix and actual fixture results

The automated laboratory fixtures were generated with the commands in the README and analyzed with the detector. The focused test suite verifies packet construction and alert classes.

Experiment	Input	Observed result	Evidence
1 Normal traffic	baseline advertisement from allowlisted printer	no unauthorized alert when allowlists match	`pcap_files/normal/normal.pcap`
2 Unauthorized discovery	baseline from `172.28.77.40`	unauthorized advertisement alert	`evidence/detection_results/unauthorized.json`
3a Invalid endpoint	port `9999`	suspicious endpoint alert	`pcap_files/attacks/invalid-port.pcap`
3b Repetition	same advertisement three times	repeated-discovery alert	`pcap_files/attacks/repeated.pcap`
3c Malformed fields	non-URI payload	malformed-discovery alert	`pcap_files/attacks/malformed.pcap`
4 Detection	all attack fixtures	JSON exports contain expected alert classes	`evidence/detection_results/`

These are reproducible Scapy laboratory results; a real Docker exchange and packet screenshots must be captured on a host with Docker network capture permissions before final submission.

4. Mitigation

The implemented mitigation is a `cups-browsed` host allowlist containing only `172.28.77.30`, applied with `docker compose -f docker-compose.yml -f docker-compose.mitigated.yml up -d`. The live pre-mitigation test (see `evidence/logs/docker-victim.log`) showed the victim accepting both advertisements and attempting IPP requests, including the unauthorized `172.28.77.40:9999` endpoint. Re-run that packet after the override produced `browse packet from 172.28.77.30 disallowed` and no accepted printer entry (see `evidence/logs/docker-mitigated-victim.log`). Network restriction of UDP/631, security patches, and disabling unnecessary `cups-browsed` functionality remain recommended defense-in-depth.

5. False positives, missed attacks, and limitations

An authorized printer using a changed address or non-631 service can trigger an alert, so allowlists must be maintained. Encrypted or fragmented traffic, alternate discovery formats, and packets without an IP layer are outside this small parser and may be missed. The detector is an investigation tool, not a replacement for patched CUPS, segmentation, or host firewall policy.

6. Ethical considerations

All packets are intended for the private Docker network only. No public hosts, physical printers, or third-party services should be targeted. The tool's `--dry-run` mode supports evidence generation without transmission, and live testing should be performed only with explicit authorization.

7. Requirements traceability

The maintained requirement-by-requirement table is in [../README.md](../README.md#requirements-traceability);
it maps each assignment item to implementation, verification, evidence, and remaining work.